

METHOD OVERLOADING AND METHOD OVERRIDING

Aim:

To understand and implement method overloading and method overriding.

PRE LAB EXERCISE

QUESTIONS

- ✓ What is method overloading?

Method overloading means using the same method name with different parameters (number, type, or order of parameters) in the same class.

Purpose:

To perform similar operations with different inputs.

Simple: Same method name, different arguments.

- ✓ What is method overriding?

Method overriding means a child class provides its own implementation of a method that is already defined in the parent class.

Purpose:

To change or customize the parent class behavior.

Simple: Child class changes the parent class method.

- ✓ Difference between overloading and overriding.

Method Overloading:

- Same method name with different parameters (number, type, or order).
- Occurs within the same class.
- Achieves compile-time polymorphism.
- Inheritance is not required.

Method Overriding:

- Same method name with the same parameters.
- Occurs between parent and child classes.
- Achieves runtime polymorphism.
- Inheritance is required.

IN LAB EXERCISE

Objective:

To demonstrate compile-time and runtime polymorphism.

PROGRAMS:

1.Student Result System (Method Overriding)

Description:

- Base class Student has method displayResult().
- Subclasses UGStudent and PGStudent override the method to show different grading systems.

Code :

```
import java.util.Scanner;
```

```
// Base class
```

```
class Student {
```

```
    String name;
```

```
    void displayResult() {
```

```
        System.out.println("Student Result");
```

```
    }
```

```
}
```

```
// UG Student subclass
```

```
class UGStudent extends Student {
```

```
    int marks;
```

```
    UGStudent(String n, int m) {
```

```
        name = n;
```

```
        marks = m;
```

```
    }
```

```
@Override
```

```
void displayResult() {  
    double percentage = (marks / 100.0) * 100;  
    System.out.println("UG Student: " + name);  
    System.out.println("Marks: " + marks);  
    System.out.println("Percentage: " + percentage + "%");  
}  
}
```

// PG Student subclass

```
class PGStudent extends Student {  
    double gpa;
```

```
    PGStudent(String n, double g) {  
        name = n;  
        gpa = g;  
    }
```

@Override

```
void displayResult() {  
    System.out.println("PG Student: " + name);  
    System.out.println("GPA: " + gpa + " / 10");  
}  
}
```

// Main class

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        // Input for UG student  
        System.out.print("Enter UG Student Name: ");
```

```

String ugName = sc.nextLine();
System.out.print("Enter UG Student Marks (out of 100): ");
int ugMarks = sc.nextInt();
sc.nextLine(); // consume newline

// Input for PG student
System.out.print("Enter PG Student Name: ");
String pgName = sc.nextLine();
System.out.print("Enter PG Student GPA (0-10): ");
double pgGpa = sc.nextDouble();

// Create objects
Student s1 = new UGStudent(ugName, ugMarks);
Student s2 = new PGStudent(pgName, pgGpa);

System.out.println("\n--- Student Results ---");
s1.displayResult();
System.out.println();
s2.displayResult();

sc.close();
}
}

```

OUTPUT:

Sample Input:

Enter UG Student Name: Ram

Enter UG Student Marks (out of 100): 85

Enter PG Student Name: Ravi

Enter PG Student GPA (0-10): 9.2

Output:

--- Student Results ---

UG Student: Ram

Marks: 85

Percentage: 85.0%

PG Student: Ravi

GPA: 9.2 / 10

```
<terminated> Exercisee [Java Application] C:\Users\visha\.p2\po
```

```
Enter UG Student Name: Ram
```

```
Enter UG Student Marks (out of 100): 90
```

```
Enter PG Student Name: Raja
```

```
Enter PG Student GPA (0-10): 8
```

```
<terminated> Exercisee [Java Application] C:\Users\visha\.p2\po
```

```
|  
--- Student Results ---
```

```
UG Student: Ram
```

```
Marks: 90
```

```
Percentage: 90.0%
```

```
PG Student: Raja
```

```
GPA: 8.0 / 10
```

2. Calculator Program (Method Overloading)

Description:

Create a Calculator class with multiple add() methods to calculate:

- Addition of 2 integers
- Addition of 3 integers
- Addition of 2 double numbers

Code:

```
import java.util.Scanner;
```

```
class Calculator {
```

```
    int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    int add(int a, int b, int c) {
```

```

        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Calculator calc = new Calculator();

        System.out.print("Enter two integers: ");
        int x = sc.nextInt();
        int y = sc.nextInt();
        System.out.println("Sum of two integers: " + calc.add(x, y));

        System.out.print("Enter three integers: ");
        int p = sc.nextInt();
        int q = sc.nextInt();
        int r = sc.nextInt();
        System.out.println("Sum of three integers: " + calc.add(p, q, r));

        System.out.print("Enter two decimal numbers: ");
        double a = sc.nextDouble();
        double b = sc.nextDouble();
        System.out.println("Sum of two doubles: " + calc.add(a, b));

        sc.close();
    }
}

```

Output:

Sample Input:

Enter two integers: 10 20

Enter three integers: 5 10 15

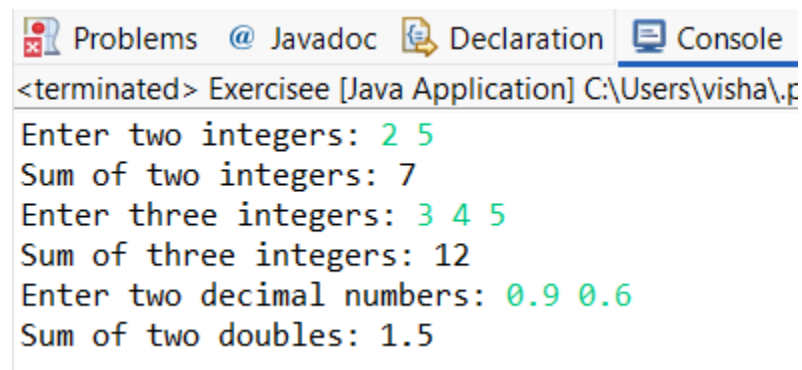
Enter two decimal numbers: 2.5 3.5

Output:

Sum of two integers: 30

Sum of three integers: 30

Sum of two doubles: 6.0



```
<terminated> Exercisee [Java Application] C:\Users\visha\p
Enter two integers: 2 5
Sum of two integers: 7
Enter three integers: 3 4 5
Sum of three integers: 12
Enter two decimal numbers: 0.9 0.6
Sum of two doubles: 1.5
```

POST LAB EXERCISE

- ✓ Is return type important in method overloading and method overriding?

Method Overloading: Return type alone is not important. Parameters must be different.

Method Overriding: Return type should be the same or compatible with the parent method.

- ✓ Can you overload a method by changing only the return type?

No.

Changing only the return type is not enough.

The parameter list must be different, otherwise Java gives a compile-time error.

- ✓ Can static methods be overridden? Can they be overloaded?

- Static methods cannot be overridden (they belong to the class, not the object).
- Static methods can be overloaded (same method name with different parameters).

- ✓ Can a method be overridden if the parameter list is different?

No.

If the parameter list is different, it is method overloading, not overriding.
For overriding, the method name and parameters must be the same.

Result:

Thus the method overloading and overriding concepts were implemented and executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		